

Практическая работа №5.

Построение графиков функций двух переменных на языке Python

Цель работы. Целью работы является изучение средств построения графиков функций двух переменных на Python.

Краткие теоретические сведения

Построение графиков двух переменных является более общей задачей по отношению к построению графиков функций одной переменной. Такая задача известна также как задача построения графиков поверхностей и встречается довольно часто. Функция при этом может задаваться формулой, например, такой

$$z(x,y)=2\sin(2\pi x)\cdot\cos(1.5\pi y)\cdot(1-x^2)\cdot y\cdot(1-y), \quad (5.1)$$

а может быть представлена уже готовым массивом данных. В языке Python для визуализации графиков функций двух переменных также часто всего используется библиотека `matplotlib`. Для ознакомления с типовыми приемами ее использования для построения графиков функций двух переменных можно порекомендовать [1].

В качестве примера рассмотрим задачу построения графика функции, заданной формулой (5.1), на отрезке изменения аргумента x от минус 1 до 1, а аргумента y от 0 до 1. Текст программы приведен в листинге 5.1.

Листинг 5.1. Программа, осуществляющая построение графика функции, заданной формулой (5.1)

```
import numpy as np
import matplotlib.pyplot as plt

points = 500
x, y = np.meshgrid(np.linspace(-1, 1, points), \
                    np.linspace(0, 1, points))
z = 2*np.sin(2*np.pi*x)*x*np.cos(1.5*np.pi*y)*(1-x**2)*y*(1-y)

# -----
# для построения графика создаем объект fig
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(x, y, z) # расчет данных для построения
ax.set_title('каркасная поверхность')
# -----
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

Для генерации последовательностей значений аргументов x и y использована команда `linspace(...)`, а формирование двумерной сетки аргумента функции производится на основе этих сгенерированных одномерных последовательностей чисел. Расчет данных для построения графика происходит с помощью метода `plot_wireframe`, определенного для созданного ранее в программе *контейнера* `ax` для объекта `fig` типа “figure”. Для добавления названия диаграммы используется метод `set_title`, для меток на осях методы `xlabel` и `ylabel` библиотеки `matplotlib`.

Как и в случае построения графиков функций одной переменной выбор количества точек, по которому строится график функции, является ответственным этапом. Рекомендации, сформулированные по этому поводу в методических указаниях к практической работе №3, полностью справедливы и здесь: график непрерывной функции не должен быть “рваным” или “угловатым”.

В результате запуска программы, текст которой приведен в листинге 5.1, будет выведено окно, в котором отображен график функции двух аргументов, представленный в формате каркасной поверхности (см. рис. 5.1).

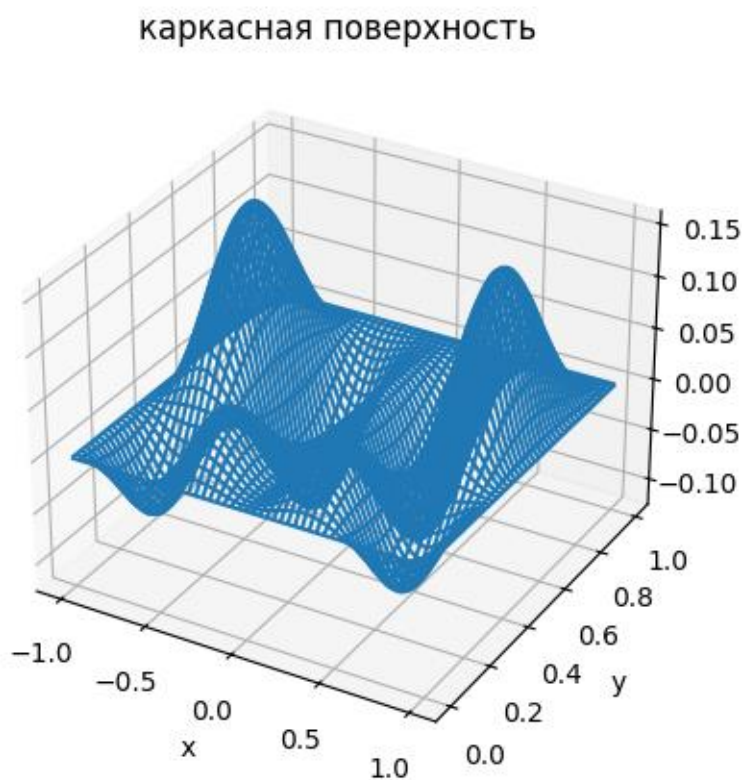


Рис. 5.1. График функции двух аргументов, заданной формулой (5.1), построенный посредством программы из листинга 5.1

Выведенное окно с графиком является интерактивным – можно с помощью мышки повернуть систему координат на любой угол с целью

достижения наилучшего ракурса при взгляде на построенный график поверхности.

Нужно заметить, что в общем случае вариант построения функции посредством каркасной поверхности не является самым наглядным. В листингах 5.2 и 5.3 приведены фрагменты кода, которые можно вставить вместо фрагмента кода, расположенного между строками комментария из символов #----, чтобы получить графики той же функции двух аргументов, где для улучшения восприятия данных графика используется раскраска псевдоцветом. В тексте программы листинга 5.2 для добавления цветов для каждой клетки, которые зависели от значения функции в соответствующей окрестности, используется аргумент `cmap` (от англ. “color map” – цветовая палитра), который позволял «раскрашивать» каждую клетку отдельным цветом. Чтобы обеспечить лучшее восприятие данных из полученного графика, в программе использована функция `colorbar`, которая дополнительно отображает цветовую шкалу, устанавливающую соответствие между цветом и величиной самой функции $z(x,y)$ в каждой окрестности ее аргументов, рядом с построенным графиком. Палитру 'viridis' можно заменить на другую из предусмотренных синтаксисом используемого метода.

Листинг 5.2. Фрагмент кода, обеспечивающий построение графика поверхности с использованием цветовой шкалы

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(x, y, z, cmap='viridis')
fig.colorbar(surf)
ax.set title('график поверхности с цветовой шкалой')
```

Листинг 5.3. Программа, осуществляющая построение графика функции двух аргументов методом контурной плоскости (contour chart)

```
color_levels = 100 # задает количество разных цветов
plt.contourf(x, y, z, color_levels)
plt.title('график контурной плоскости с цветовой шкалой')
plt.colorbar()
```

В листинге 5.3 используется, пожалуй, самый наглядный способ отображения функций двух переменных – отображением поверхности на декартовой плоскости, где величина функции при каждой паре аргументов x и y передается только цветом в соответствии с используемой цветовой палитрой, примерно так же, как на географических картах принято отображать рельеф. В результате выполнения программы с данным фрагментом кода (вставляется в листинг 5.1 между строками комментариев #----) будет выведено окно, содержимое которого приведено на рис. 5.3.

график поверхности с цветовой шкалой

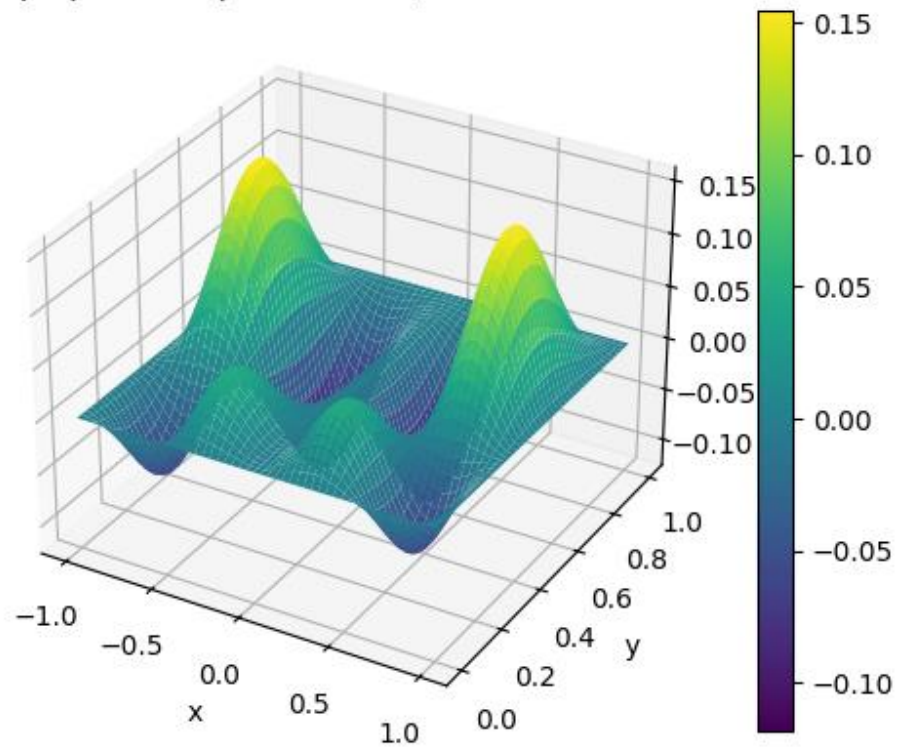


Рис. 5.2. График функции, построенный программой из листинга 5.2

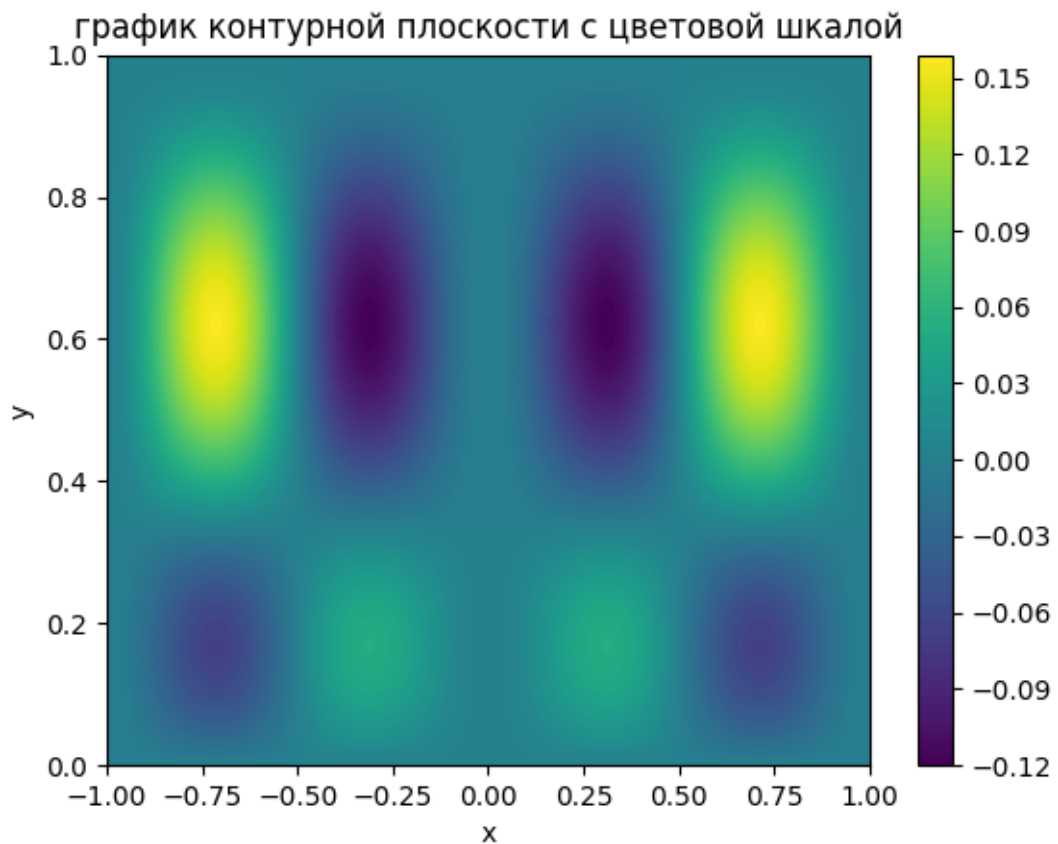


Рис. 5.3. Представление функции двух аргументов в виде контурной плоскости с использованием раскраски псевдоцветом.

Переменная `color_levels` в программе определяет количество цветов, которые используются для отображения данных на графике. Если исключить этот четвертый параметр из аргументов метода `contourf`, то количество цветов будет автоматически выбрано таким, чтобы обеспечить наилучшее отображение данных. Однако на практике это обычно существенно обедняет результат. При желании подробнее изучить варианты параметризации метода `contourf` можно обратиться к документации (на англ. языке: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.contourf.html) или дополнительным обучающим материалам (например, на рус. языке <https://jenyay.net/Matplotlib/Contour>).

Так же, как и в случае с построением графиков функций одной переменной, при построении графиков функций двух аргументов часто приходится обращаться к логарифмическому масштабу по одной или нескольким осям. Например, если построить график функции $z(x, y) = (5-x)/y$ в диапазоне изменений x от 0 до 10, и y в диапазоне от 0.01 до 10, в линейном масштабе, то результат будет совершенно неудовлетворительным (см. рис. 5.4): если на графике поверхности еще виден резкий “заворот” поверхности в области малых значений y , то на контурной плоскости ширина соответствующей полосы внизу графика настолько мала, что зачастую даже не может быть замечена.

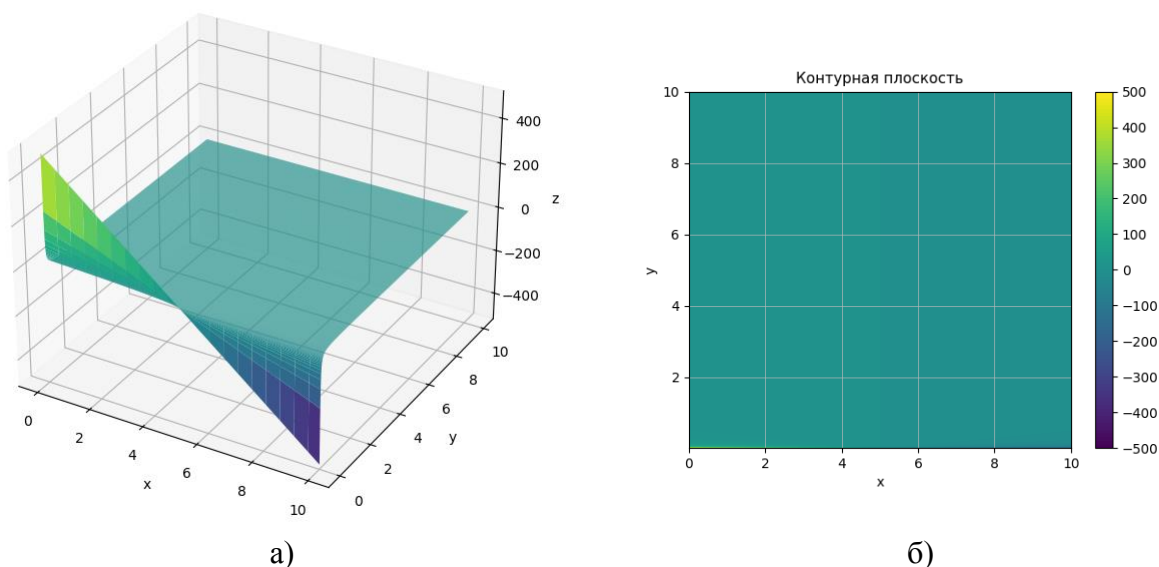


Рис. 5.4. График функции $z(x, y) = (5-x)/y$, построенный в линейном масштабе: а) график поверхности с цветовой шкалой, б) график контурной плоскости

Если учесть, что в данном случае при уменьшении аргумента y значение z резко возрастает (по модулю), то крайне желательно использовать по оси y логарифмический масштаб. Для оси z , увы, в данном случае логарифмический масштаб использовать нельзя, поскольку при некоторых значениях x функция принимает нулевые и отрицательные значения. Полезно изучить программу, текст которой приведен в листинге 5.4, чтобы научиться

пользоваться инструментарием, который необходим для построения графиков поверхностей с использованием логарифмического масштаба.

Листинг 5.4. Программа, осуществляющая построение графика функции двух аргументов в полулогарифмическом масштабе

```
import numpy as np
import matplotlib.pyplot as plt

points_x = 100 # кол-во точек по оси x
points_y = 5000 # кол-во точек по оси y
x, y = np.meshgrid(np.linspace(0, 10, points_x),
                   np.linspace(0, 10, points_y))
y = np.where(np.abs(y) > 0.01, y, 0.01)
z = (5-x)/y # заданная функция
log_y = np.log10(y)

# строим график поверхности в полулог. масштабе (по оси y):
fig, ax = plt.subplots(subplot_kw={'projection': '3d'},
                       figsize=(10, 8))
surf = ax.plot_surface(x, log_y, z, cmap='viridis',
                       rstride=5, cstride=5)

# ставим метки по осям:
y_ticks = np.array([0.01, 0.1, 1])
ax.set_yticks(np.log10(y_ticks))
ax.set_yticklabels(y_ticks)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
#plt.tight_layout()
plt.show() # построили график поверхности

# строим график конт. плоскости в полулог. масштабе (по оси y):
color_levels = 120
CS = plt.contourf(x, y, z, color_levels)

# задаем цветовую шкалу красиво:
cbar = plt.colorbar(CS)
ticks = np.linspace(z.min(), z.max(), num=10+1)
cbar.set_ticks(ticks)

plt.title('Контурная плоскость', fontsize=11)
plt.xlabel('x')
plt.ylabel('y')
plt.yscale('log') # лог. масштаб по оси y
# наносим сетку:
plt.grid(which = 'major', color = '#bbbbbb',
         linewidth = 0.5)
plt.grid(which = 'minor', color = '#999999',
         linewidth = 0.5,
         linestyle = ':')
plt.show() # построили контурную диаграмму
```

Результат работы программы приведен на рис. 5.5.

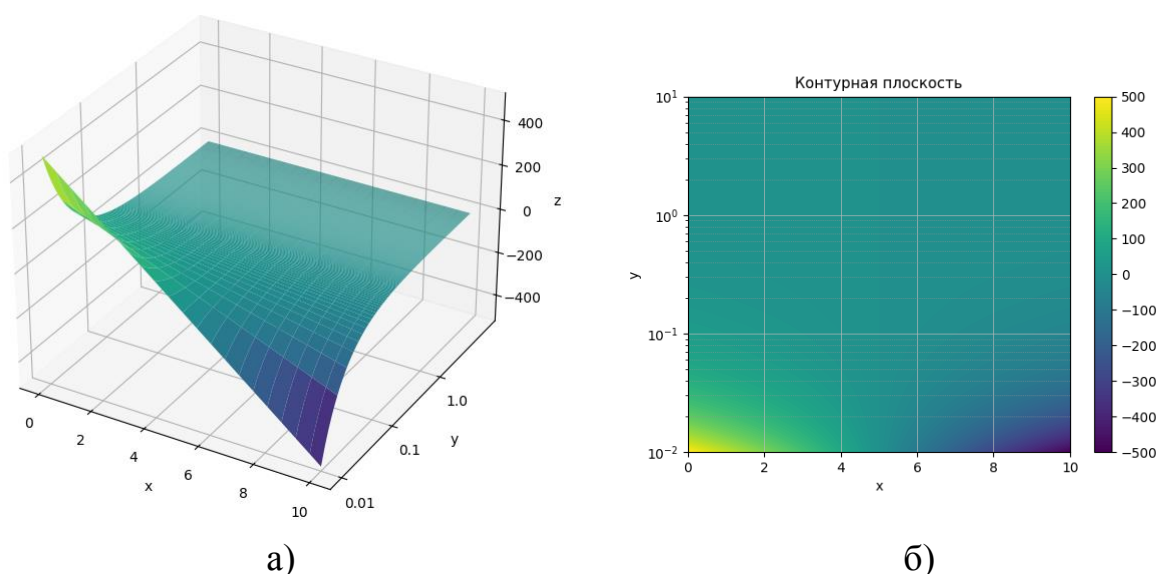


Рис. 5.5. График функции $z(x, y) = (5-x)/y$, построенный в полулогарифмическом масштабе: а) график поверхности с цветовой шкалой, б) график контурной плоскости

Нельзя, конечно, утверждать, что полученный график идеален, но он определенно лучше, чем построенный в линейном масштабе. При необходимости улучшить наглядность представления информации, можно, ориентируясь на общий вид графика, строить графики локальных областей в области интереса. Например, исходя из свойств наблюдаемых графиков можно предположить, что в данном случае целесообразно ограничить диапазон вывода по переменной y от 0.01 до 0.1 (см. рис. 5.6).

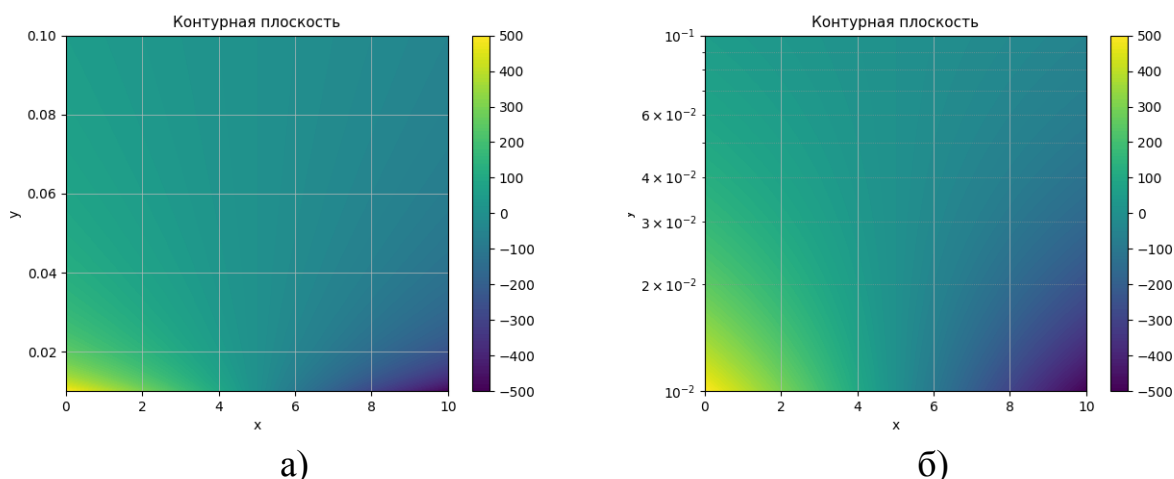


Рис. 5.6. Графики контурной плоскости для локальной области функции $z(x, y) = (5-x)/y$: а) в линейном масштабе, б) в полулогарифмическом масштабе (по оси y)

Даже при таком примитивном подходе к улучшению информативности графика логарифмический масштаб может обеспечить некоторое преимущество перед линейным, поскольку в общем случае обеспечит лучшее растяжение в области малых значений аргумента, особенно, если наносить на график контурные линии уровня.

Варианты заданий

Данная практическая работа предполагает выполнение двух заданий. В обоих заданиях следует написать программы на языке Python для построения графиков функции двух переменных $z(x,y)$, которую каждый обучающийся формирует из функций, использованных при выполнении практической работы №4.

1) Задание №1: осуществить построение графика поверхности и контурной плоскости для функции $z(x,y)=y_1(x) \cdot y_2(y)$ в диапазоне изменения аргументов: x от 0 до 10, y – от ℓ (найденно в прошлой работе) до 10. Для построения рекомендуется опираться на примеры программ, приведенных в листингах 5.1 – 5.4.

2) Задание №2: получить график контурной плоскости для функции $z(x,y)=y_1(x) \cdot y_1(y)$ в диапазоне изменения аргументов: x и y от 0 до 10, с нанесенными на него линиями уровня, для двух уровней – нижнего и верхнего, рассчитываемых по формулам $\text{level_high}=\max(z)-0.25 \cdot (\max(z)-\min(z))$ и $\text{level_low}=\min(z)+0.25 \cdot (\max(z)-\min(z))$ соответственно.

В обоих заданиях обязательно наличие масштабной сетки на графиках.

Каждый обучающийся должен принять самостоятельное решение об использовании логарифмического масштаба по одной или обеим осям графика, а также обосновать размеры областей для построения локальных областей графиков.

Для выполнения второго задания рекомендуется ознакомиться с материалами ресурса <https://jenyay.net/Matplotlib/Contour>. Кроме того, в программе понадобится использовать средства, обеспечивающие форматирование вывода чисел, чтобы содержимое линий уровня корректно отображалось. Например, для рассмотренной в методических указаниях функции (5.1) полученный результат должен иметь вид, показанный на рис. 5.7. В случае, когда числа на графике не отображаются на графике, построенном в заданном диапазоне изменения аргументов x и y , следует построить графики локальных области функции, подобрав необходимые ограничения на диапазон изменения аргументов. Кроме того, исходя из свойств функций, входящих в задание, необходимо обязательно рассмотреть возможность использования логарифмического масштаба по оси y .

Порядок выполнения работы

1. Получить вариант задания у преподавателя и убедиться в правильном понимании задания и критериев его оценивания.
2. Используя приведенные в метод. указаниях листинги программ, написать и отладить программу для решения задачи №1, запустить ее на выполнение в IDE и сохранить полученные результаты.
3. Написать и отладить программу для решения задачи №2, запустить ее на выполнение в IDE и сохранить результаты. Следует получить как общий вид графика, так и – при недостаточной наглядности графика общего вида – графики локальных областей, представляющей интерес, так чтобы можно было наглядно оценить геометрию построенных линий уровня.
4. Получить результаты работы программ и подвергнуть их критическому анализу.

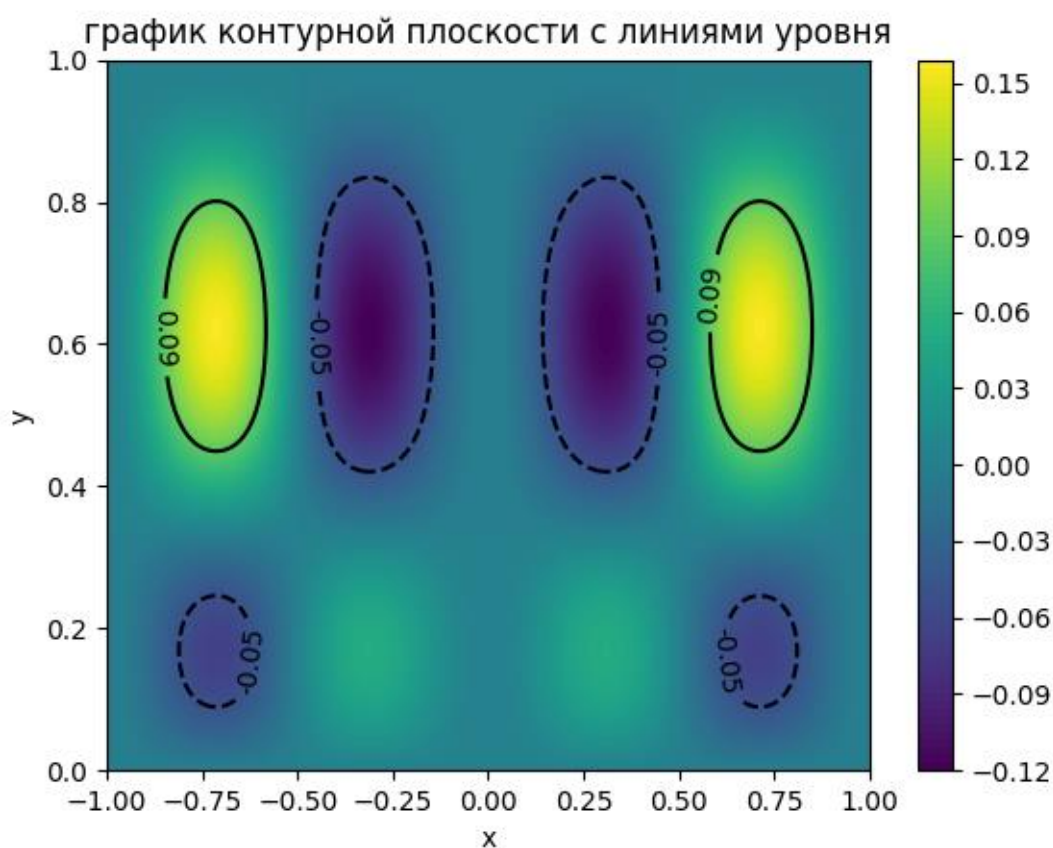


Рис.5.7. Пример ожидаемого результата при выполнении второго задания (внимание: здесь диапазоны изменения аргументов x и y не такие как в заданиях, кроме того, не нанесена координатная сетка).

Требования к содержанию отчета

1. Цель работы.
2. Подробный формулировки полученных заданий, с конкретизацией формулы для $z(x,y)$ в тексте отчета в обоих заданиях. Рекомендуется для каждой задачи отвести свой раздел в пределах отчета.
3. Программы с необходимыми комментариями, а также описание наиболее важных ее фрагментов в основном тексте отчета, с обоснованием всех принятых решений.
4. Полученные графики при решении поставленных задач: минимум 4 рисунка: три для задания №1 и один для задания №2. Хотя при недостаточной наглядности графиков общего вида, возможно, понадобится еще несколько рисунков для локальных областей.
5. Выводы по работе.
6. Список использованной литературы.

При оформлении отчета о практической работе следует придерживаться требований правил, размещенных на сайте ГУАП.

Рекомендуемая литература

1. Использование библиотеки Matplotlib. Как рисовать трехмерные графики // URL: <https://jenyay.net/Programming/Python3d>
2. Python для анализа данных. Библиотека matplotlib // URL: https://mipt-stats.gitlab.io/courses/python/06_matplotlib.html
3. «Укус Питона» – «A Byte of Python» по-русски / Swaroop С.Н., пер. с англ. В. Смоляр. ver.2.02, apr 2020. – Sphinx. // свободно распространяется по лицензии Creative Commons Attribution-Share Alike 3.0 Unported, 2020.